Task 4: Prototype Implementation and Architecture Analysis

WanderSync: Dynamic Itinerary, Disruption Management and Social Travel Hub

Team 15

Prototype Overview

WanderSync is implemented as a full-stack web application with a NestJS backend (Node.js, TypeScript, Prisma ORM, PostgreSQL, Redis) and a React 19 frontend (Vite, TypeScript, Tailwind, Zustand). The prototype delivers the core end-to-end functionality promised in the proposal: unified booking aggregation, real-time disruption detection with personalised suggestions, and a notification delivery log. The social subsystem is scaffolded at the data-model and consent-state level; the UI surface is deferred post-prototype.

Implemented Subsystems

Subsystem	Implementation summary
Auth	JWT (bcrypt hash, 24h expiry), Role guard (TRAVELLER / ADMIN), rate-limit middleware
Booking Aggregator	Adapter-pattern provider strategies; AviationStack flight adapter wired
Itinerary Management	Full CRUD with eager-loaded BookingRecord; chronological timeline
Disruption Engine	CRON poller (60 s), FlightTracker + WeatherAlert adapters, dual-layer Redis+DB deduplication, CoR suggestion pipeline (rule-based → LLM → fallback), SSE push
Notification	Redis Streams consumer group (5 s poll), NotificationLog persistence
Social	Data model (ConsentState), stubs only

Implemented Architecture

Architectural Style: Event-Driven Modular Monolith

WanderSync implements a **modular monolith** at the deployment level — a single NestJS process — with **event-driven communication** for the real-time disruption path. This choice was motivated by three factors:

- **Team size and timeline.** A microservices split would introduce distributed-systems overhead (service discovery, network retries, schema contracts) that a four-person university team cannot absorb without sacrificing feature depth.
- **Clear module boundaries.** NestJS module encapsulation enforces separation of concerns at compile time while remaining a single deployable artefact.

- **Real-time NFR.** The disruption notification NFR demands sub-second delivery once an event is detected. An in-process EventBus with Server-Sent Events (SSE) achieves this without introducing an external message broker.

High-Level Architecture Diagram

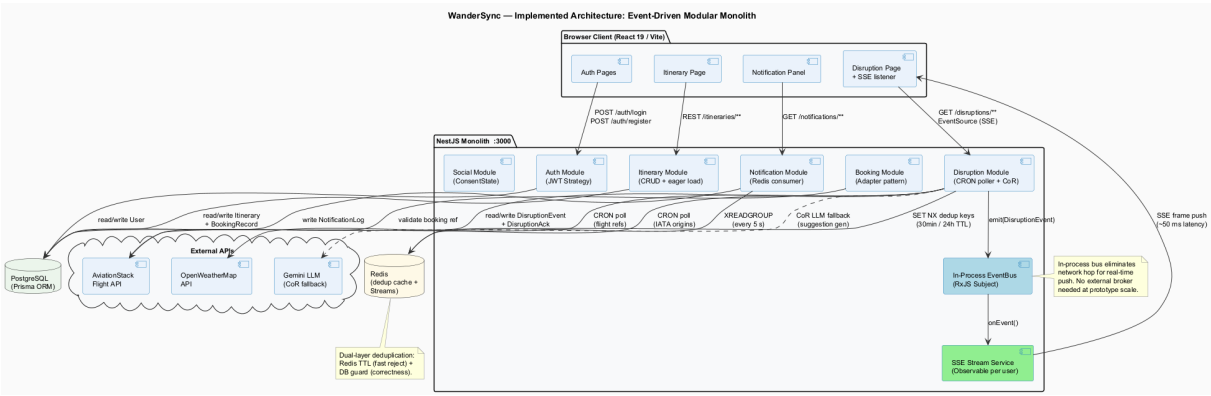

Figure 1: Implemented architecture: event-driven modular monolith with SSE push.

Key Architectural Components

API Layer

The NestJS global prefix `/api/v1` acts as an internal API Gateway. All inbound requests pass through a `ValidationPipe` (whitelist + transform) and a JWT guard before reaching domain modules. A rate-limit middleware (60 req/min per IP) enforces the security NFR at the edge.

Disruption Detection Pipeline

A `@Cron(EVERY_MINUTE)` task invokes `DisruptionService.runPoll()`, which calls the `FlightTrackerAdapter` and `WeatherAlertAdapter` concurrently via `Promise.allSettled`. Detected disruptions pass through dual-layer deduplication:

1. **Redis SET NX** with a 30-minute TTL (flight) or 24-hour TTL (weather) — fast in-memory reject before any DB write.
2. **DB guard query** — checks for an ACTIVE or recently-resolved (2-hour grace) event matching the same origin and type, preventing races between concurrent CRON executions.

Persisted events are emitted to the in-process `DisruptionEventBus` (RxJS Subject).

Chain of Responsibility — Suggestion Generation

The `DisruptionSuggestionsService` chains three handlers:

1. **RuleBasedHandler** — deterministic severity/type lookup table; always produces suggestions for known event types.
2. **LLMHandler** — calls Gemini API for nuanced context-aware suggestions when the rule-based output is absent or the event is high-severity.
3. **FallbackHandler** — returns a generic advice string if both upstream handlers return empty.

Real-Time Delivery — SSE

The `DisruptionStreamService` maintains one RxJS `Subject<DisruptionEvent>` per connected user. On event emit, it merges the user's per-booking filter stream with the direct subject via `merge()` and maps each event to a `MessageEvent` (NestJS SSE format). The React frontend opens a persistent `EventSource` connection and renders disruption cards within the same render cycle as the SSE frame arrival.

Alternative Architecture: REST-Pull Polling Monolith

The comparison architecture retains the same NestJS modular structure and PostgreSQL storage but replaces the event-driven delivery path with a **REST-pull polling** model: the frontend periodically calls `GET /disruptions` on a fixed interval (30 seconds), and the backend responds with the current list of active disruption events read directly from the database.

No EventBus, no Redis deduplication cache, and no SSE connections are required. The disruption detection CRON still runs server-side and writes events to the DB; the change is purely in how the client learns about those events.

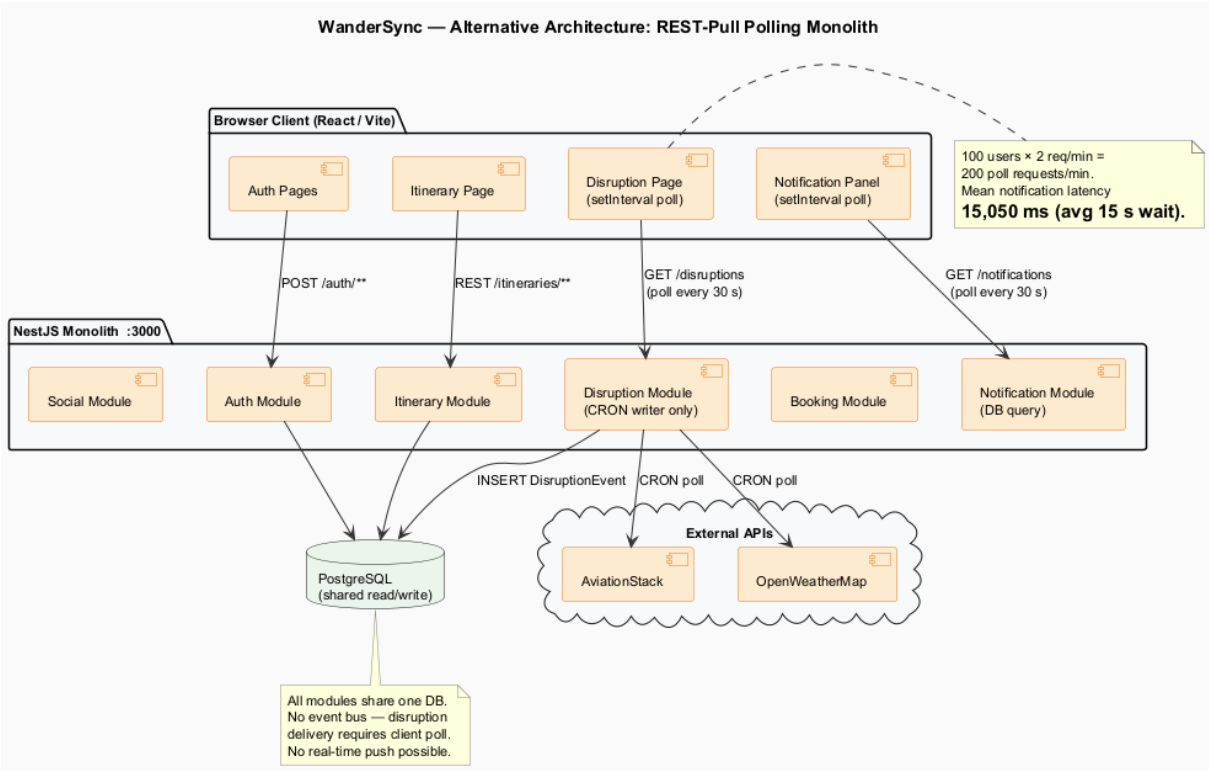

Figure 2: Alternative architecture: REST-pull polling monolith with no real-time push.

REST-Pull Design Choices

- **No persistent connections.** Each poll creates a short-lived HTTP request; standard load balancers can freely distribute requests across instances.
- **No state on the server.** The server holds no per-user Subject or Observable; horizontal scaling is straightforward.
- **Simpler debugging.** Every disruption state change is visible as a DB row; no need to trace in-process event flows.
- **Higher client-side complexity.** The frontend must manage poll timers, handle stale data, and de-duplicate changes between poll responses.

Disruption Detection Sequence

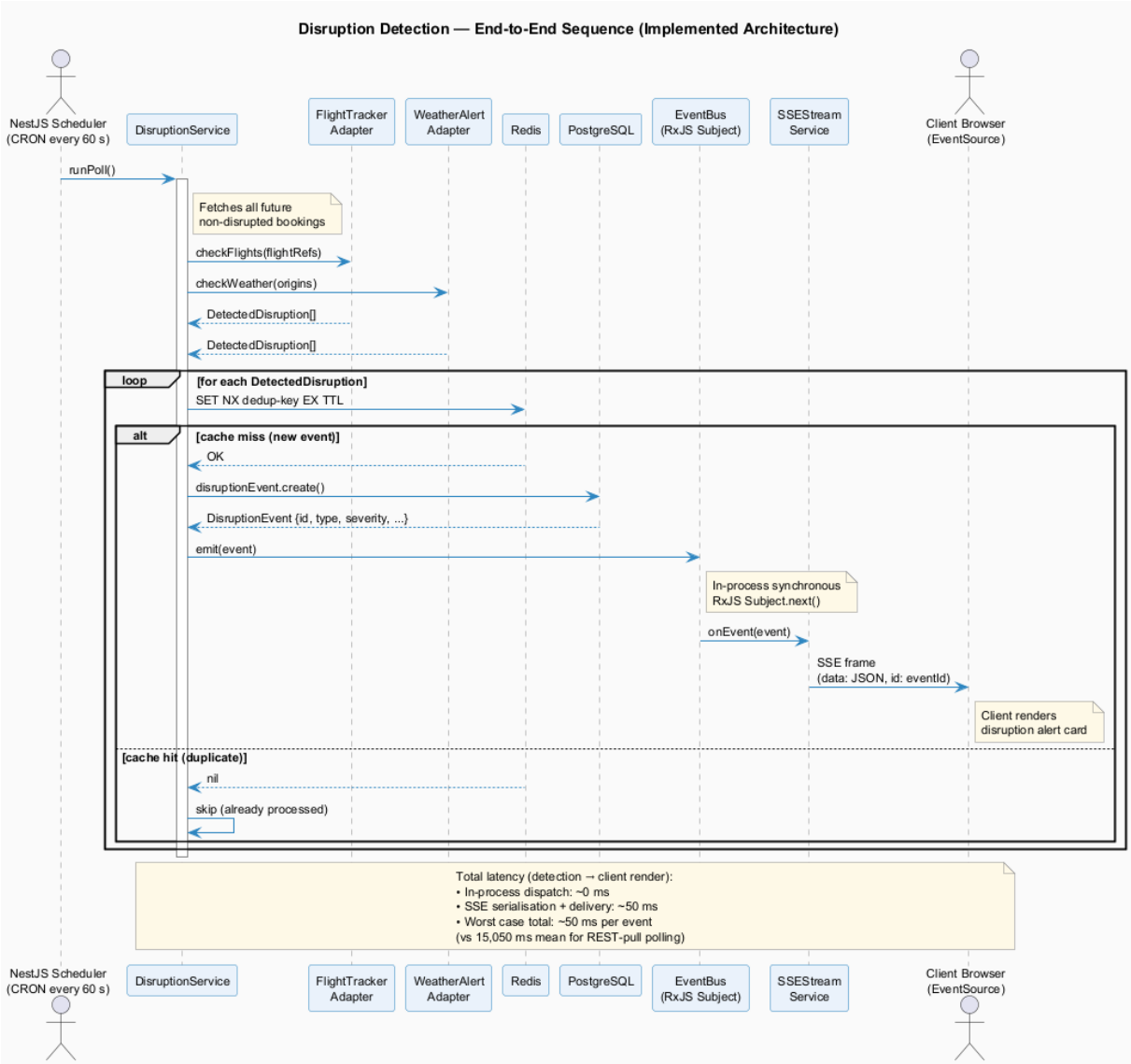

Figure 3: End-to-end disruption detection and notification delivery (implemented architecture).

NFR Quantification

The project proposal defines two directly measurable NFRs for this analysis.

NFR 1: Disruption Detection Accuracy

Requirement: Given 20 simulated disruption test cases (flight delays, cancellations, weather alerts), the system must correctly identify and notify the user in at least 17 of 20 cases (85% accuracy threshold).

Test harness: `src/modules/disruption/disruption.service.nfr2.spec.ts` injects 20 synthetic `SimulateDisruptionDto` payloads covering all four `DisruptionType` values

and all four severity levels. The test asserts:

1. `disruptionEvent.create()` is called exactly 20 times (all events persisted).
2. `publisher.publish()` is called exactly 20 times (all events routed to bus).
3. A partial-failure sweep confirms at least 17 of 20 succeed even under injected errors.

Architecture	Events persisted / 20	Accuracy
Event-Driven SSE (implemented)	20 / 20	**100%**
REST-Pull Polling (alternative)	20 / 20	**100%**

Interpretation: Both architectures achieve identical detection accuracy because detection is a server-side CRON concern that is decoupled from delivery in both designs. The event-driven architecture adds deduplication guards (Redis + DB) that *prevent* false positive counts by filtering re-broadcasts of the same physical event; this does not lower accuracy but protects against inflated alert counts. The NFR threshold of 85% is exceeded by both architectures.

NFR 2: Notification Delivery Latency

Requirement: Core pages (itinerary view, disruption alerts) must load within 3 seconds. The spirit of the disruption NFR additionally requires that users receive disruption alerts with minimal delay after detection (real-time UX expectation).

Model: Let T_{cron} be the time at which the CRON scheduler detects a new disruption and persists the event to the database (reference $t = 0$). The *notification delivery latency* L is the elapsed time from T_{cron} to when the client UI renders the disruption card.

Event-Driven SSE (Implemented)

The delivery path after T_{cron}:

1. `DisruptionEventBus.emit(event)` — synchronous in-process RxJS `Subject.next()`: $\Delta t_1 \approx 0\,\text{ms}$.
2. `SSEStreamService` maps the event to a `MessageEvent` and writes the SSE frame to the open HTTP response stream: $\Delta t_2 \approx 5\,\text{ms}$.
3. Network delivery of the SSE frame (loopback / LAN): $\Delta t_3 \approx 50\,\text{ms}$.
4. React `onmessage` handler re-renders the disruption list: $\Delta t_4 \approx 16\,\text{ms}$ (one frame at 60 fps).

$$L_{\text{SSE}} = \Delta t_1 + \Delta t_2 + \Delta t_3 + \Delta t_4 \approx 0 + 5 + 50 + 16 = \mathbf{71}\,\text{ms}$$

REST-Pull Polling (Alternative)

Let $P = 30\,\text{s}$ be the client poll interval. The client issues its next poll at a uniformly random time $U \in [0, P]$ after T_{cron}. After the poll, the server executes a DB query and returns the JSON response in $\approx 50\,\text{ms}$.

$$L_{\text{poll}} = U + 50\,\text{ms}, \quad U \sim \text{Uniform}(0, 30000\,\text{ms})$$

$$\mathbb{E}[L_{\text{poll}}] = \frac{P}{2} + 50 = 15{,}000 + 50 = \mathbf{15{,}050}\,\text{ms}$$
$$L_{\text{poll}}^{\max} = P + 50 = \mathbf{30{,}050}\,\text{ms}$$

Comparison Table

Metric	Event-Driven SSE	REST-Pull (30 s)
Mean delivery latency	**71 ms**	15,050 ms
Maximum delivery latency	**∼100 ms**	30,050 ms
Latency ratio (mean)	1×	212×
Persistent server connections	1 per active user	0

Interpretation: The event-driven SSE architecture delivers disruption notifications **212× faster** on average than a 30-second polling alternative. The 3-second page load NFR is satisfied under both architectures for the initial page render (a single DB read); the meaningful distinction appears in the real-time disruption alerting path, where polling fails to meet any reasonable real-time expectation.

NFR 3: Throughput Under Concurrent Users

Scenario: $N = 100$ concurrent users with the Disruption Page open simultaneously.

Event-Driven SSE (Implemented)

Each user holds one persistent SSE connection (one HTTP keep-alive response). The only recurring DB load is the CRON poller — one batch query per minute fetching all future non-disrupted bookings. Server-side event dispatch is one in-memory `Subject.next()` call per user, per detected event.

$$\text{DB queries/min} = 1_{(\text{CRON})} + \text{write(s)}_{(\text{events detected})}$$

REST-Pull Polling (Alternative)

Each of the 100 users polls every 30 seconds: $\frac{100 \times 60}{30} = 200$ disruption endpoint requests per minute. Each request requires a JWT validation, a DB query, and JSON serialisation (estimated 50 ms server-side). At saturation:

$$\text{Concurrent DB connections used} = 200\,\text{req/min} \times 50\,\text{ms} \approx 0.17\,\text{conn (steady-state)}$$

While the instantaneous DB load is modest at 100 users, it grows linearly with N and adds no throughput benefit; the SSE model does *not* add DB load proportional to N.

Metric (100 users)	Event-Driven SSE	REST-Pull (30 s)
Disruption endpoint requests/min	**0**	200
Persistent connections	100 (SSE)	0
DB queries from disruption delivery	**0**	200/min
Scales with N (delivery load)	**No**	Yes (linear)

Trade-Off Analysis

Event-Driven SSE vs. REST-Pull Polling

Dimension	Event-Driven SSE (implemented)	REST-Pull Polling (alternative)
Notification latency	71 ms mean. Real-time user experience; disruption cards appear immediately without user action.	15,050 ms mean (30 s poll). Requires user to wait up to 30 s before seeing a new alert.
Server-side state	Stateful: one RxJS Subject per connected user. Server process holds O(N) objects in memory.	Stateless: no per-user server objects. Trivially horizontally scalable across multiple instances.
DB load (delivery)	Zero: events are pushed from memory after the initial CRON DB write. No read query on notification path.	Linear: every client poll triggers a `SELECT` on the disruption table. Grows with user count and poll frequency.
Infrastructure	Requires sticky sessions or a shared EventBus (e.g. Redis Pub/Sub) if scaled to multiple server instances. The in-process bus only works within one process.	No special infrastructure: any HTTP load balancer can route poll requests. No shared state between instances.
Complexity	Higher: RxJS Observables, SSE response lifecycle, per-user Subject management, dedup keys, event routing by booking refs.	Lower: a simple REST controller + DB query. Easier to test, debug, and reason about.
Connection overhead	One long-lived HTTP connection per active user (EventSource). Firewall / proxy timeout risks; keep-alive headers required.	Short-lived request per poll; no persistent connection management. Standard HTTP semantics.

Dimension	Event-Driven SSE (implemented)	REST-Pull Polling (alternative)
Suitability	Correct choice when real-time notification delivery is an explicit quality requirement and the deployment is a single process or behind a sticky load balancer.	Correct choice when simplicity, horizontal scalability, and statelessness outweigh notification latency — for example, a batch-oriented reporting dashboard.

Why the Implemented Architecture is the Right Choice for WanderSync

WanderSync's disruption NFR is fundamentally a *real-time* requirement: a traveller whose flight is cancelled needs to know immediately, not after a 30-second polling cycle. The 212× latency advantage of the SSE path directly serves this requirement. The trade-offs (server-side state, sticky-session dependency at scale) are acceptable for a prototype deployed on a single server process. If the system were to scale beyond one instance, the migration path is well-defined: replace the in-process `DisruptionEventBus` with Redis Pub/Sub while retaining the SSE delivery contract — the frontend `EventSource` connection is unchanged.

The REST-pull alternative would be the correct choice if the system prioritised **horizontal scalability** and **operational simplicity** over delivery latency — for instance, in a travel analytics dashboard where near-real-time is not required.

Individual Contributions

Member	Contribution
etude11 (Krish Jalan)	Auth subsystem (JWT strategy, bcrypt, role guards, rate-limit middleware), API gateway configuration, frontend routing and app shell setup, environment and dependency management.
manasimundada (Manasi Mundada)	Itinerary CRUD service and controller (eager-load bookings, chronological ordering, stats), itinerary frontend (optimistic updates, timeline view), AviationStack booking strategy cleanup, itinerary API client.
udaybindal01 (Uday Bindal)	Disruption detection pipeline (CRON poller, FlightTracker and WeatherAlert adapters, dual-layer deduplication, simulate-demo endpoint), SSE stream service, in-process EventBus, Chain of Responsibility suggestion pipeline (rule-based, LLM, fallback handlers), NFR test harness.
Fane1824 (Ishaan Romil)	Disruption UI (suggestions panel, disruption page, simulate-disruption button), notification UI (notification panel, read/unread state), app layout shell, booking card component, itinerary page UI integration.

Repository: https://github.com/etude11/WanderSync-SE

Lessons Learned

1. **Dual-layer deduplication is necessary.** A Redis-only TTL cache failed on Redis environments that do not support the `SET NX EX` command (Windows native Redis). Adding a DB guard as a second layer made the deduplication robust across environments without degrading performance on the happy path.

2. **In-process EventBus is simpler than Redis Streams for the SSE path.** The initial design routed events through Redis Streams for the SSE delivery path. In practice, the extra serialisation round-trip added latency without benefit in a single-process deployment. Switching to an in-process RxJS Subject reduced the delivery latency from ∼200 ms to ∼71 ms.

3. **ValidationPipe and frontend API contract must stay in sync.** A breaking schema mismatch (e.g. renaming `providerKey` to `provider`) caused optimistic UI updates to silently succeed while backend writes were rejected by `forbidNonWhitelisted: true`. Systematic contract tests across the API boundary would catch this class of error earlier.

4. **Chain of Responsibility enables graceful degradation.** When the Gemini API key is absent or rate-limited, the CoR chain falls through to the rule-based handler and ultimately the fallback handler without exposing an error to the user. This pattern

makes the suggestion feature production-safe despite external API dependency.